Simon Fraser University
CMPT 295 – D100
Summer 2025
Wednesday, June 25, 2025
Midterm 1 - SOLUTION
Out of 45 marks

<u>This examination has 10 pages.</u>

Read each question carefully before answering it.

- No textbooks, cheat sheets, calculators, computers, cell phones or other materials/devices may be used.
- All assembly code must be x86-64 assembly code using ATT syntax and executable on our *target machine*.
- **Always comment your code.**
- The marks for each question are given in [ ]. Use this to manage your time:
  - One (1) mark corresponds to one (1) minute of work.
  - Do not spend more time on a question than the number of marks assigned to it.

Good luck!

1. [6 marks] Consider the following syntactically correct C main function:

```
int main(int argc, char *argv[]) {

  int x = 0xDECAF000;
  short y = 0xCAFE;
  char z = y;

  if ( x > y ) printf("Cafe\n");
  // Your modified condition is:
Here are the possible casts:
  if ( ( unsigned char ) x > y )
  if ( ( unsigned short ) x > y )
  if ( ( char ) x > y )
  if ( ( short ) x > y )

  if ( x < z ) printf("Decaf\n");
  // Your modified condition is:
Here are the possible casts:
As long as the selected cast has not been used
for the condition above:
  if ( ( unsigned char ) x < z )
  if ( ( unsigned short ) x < z )
  if ( ( char ) x < z )

  return 0;
}
```

Using only two casts (data type casts), and these two casts must be different, modify the conditions of the two conditional statements in the main function above such that only "`Cafe`" is printed on the computer monitor screen when the program executes on our *target machine*.

2. [3 marks] Consider the following syntactically correct C code fragment:

```
float aFloat = 13.75;
int sum = (int) aFloat + 0xFFFFFFFE;
```

Which value does the variable `sum` contain when the above C code fragment has executed on our *target machine*? Represent your answer as a hexadecimal number.

Answer: `0x0000000B`

3. [3 marks] Consider the following C code fragment:

```
char char1 = 101;
char char2 = _______ ;
char sumOfChar = char1 + char2;
```

Which value must be assigned to `char2` in order for the sum of `char1` and `char2` to create a positive overflow?

Possible answer: 27

Any value between 27 and 127 creates a positive overflow. No value larger than 127 is a possible answer because we can only store 8 bits in a data type `char` and these 8 bits are interpreted as [-128 .. 127] and we are only considering positive overflow.

4. [3 marks] Consider the disassembled function below. It was obtained by disassembling an executable containing this function on our *target machine*.

```
7002 :   f3 0f 1e fa              endbr64
7006 :   48 83 ec 08              sub    $0x8,%rsp
700A :   48 8b 05 d9 2f 00 00     mov    0x2fd9(%rip),%rax
7011 :   48 85 c0                 test   %rax,%rax
7014 :   74 02                    je     7016 <_init+0x18>
7016 :   ff d0                    callq  *%rax
7018 :   48 83 c4 08              add    $0x8,%rsp
701C :   c3                       retq
```

As you can see, six (6) of the memory addresses are missing. Complete the left column of the above disassembled function by adding these six (6) missing memory addresses.

5. [Total marks: 9] Consider the following function `mystery` written in x86-64 assembly code, where the parameter `x` is in the register `%edi` and the parameter `y` is in the register `%esi`:

```
      .globl  mystery                          Line 1
mystery:                                       Line 2
      endbr64                                  Line 3
      testl   %esi,   %esi                     Line 4
      jle     .L4                              Line 5
      movl    $0,     %eax                     Line 6
      movl    $1,     %edx                     Line 7
.L3:                                           Line 8
      imull   %edi,   %edx                     Line 9
      addl    $1,     %eax                     Line 10
      cmpl    %esi,   %eax                     Line 11
      jne     .L3                              Line 12
.L1:                                           Line 13
      movl    %edx,   %eax                     Line 14
      ret                                      Line 15
.L4:                                           Line 16
      movl    $1,     %edx                     Line 17
      jmp     .L1                              Line 18
```

a. [3 marks] If we call this function as follows: `mystery( x, y )` where `x = 4` and `y = 3`, which value will it return?

Answer: $4^3 = 64$

a. [2 marks] If you were to replace the two occurrences of the label `.L3`, in the `mystery` function, listed on the previous page, with a more descriptive label name, which label name would you choose? Write this more descriptive label name below:

Possible Answer: Loop

b. [2 marks] If you were to replace `Line 6` in the `mystery` function, listed on the previous page, with another equivalent x86-64 instruction, i.e., an x86-64 instruction that would produce the same result, but is not a `mov*` instruction, which instruction would you choose? Write this entire x86-64 instruction below:

Possible Answers:
```
xorl    %eax,   %eax
subl    %eax, %eax
imull   $0, %eax
andl    $0, %eax
```

c. [2 marks] If you were to replace `Line 10` in the `mystery` function, listed on the previous page, with another equivalent x86-64 instruction, i.e., an x86-64 instruction that would produce the same result, but is not a `add*` instruction, which instruction would you choose? Write this entire x86-64 instruction below:

Possible Answers:
```
incl    %eax
leal    1(%eax), %eax
subl    $-1, %eax
```

6. [Total marks: 12] Consider a floating point number encoding scheme based on the IEEE 754 standard for floating point representation and defined as follows:

   - It uses 8 bits.
   - There is one sign bit `s`.
   - The `exp` can be any number in the range [0 .. 31].

   This means that k = 5 and that n = 2 -> s + k + n = 8 bits!

   a. [2 marks] Compute the bias of this IEEE-like encoding scheme described above and show your work:

      Answer: k = 5 and bias = $2^{k-1} - 1 = 2^{5-1} - 1 = 2^4 - 1 = 16 - 1 = 15$

   b. [1 mark per cell for a total of 7 marks] Complete the table found on the next page with bit patterns representing the real numbers listed in the left column using the IEEE-like encoding scheme described above.

   c. [3 marks] Can **`1.875 x 2^-15`** be encoded exactly (not approximated) as a normalized number using the IEEE-like encoding scheme described above? Explain and show why or why not.

      Possible answer 1:

      `R2B(.875) = 0.5 + 0.25 + 0.125`
      `= 1/2 + 1/4 + 1/8`
      `= 0.1₂ + 0.01₂ + 0.001₂`
      `= 0.111₂`
      `R2B(1.875) = 1.111₂`
      `So,` **`1.875 x 2^-15 =`** `1.111₂` **`x 2^-15`**

      We need a `frac` of 3 bits to exactly encode the above number using the IEEE-like encoding scheme described in this question. Yet, the IEEE-like encoding scheme described in this question has a `frac` of 2 bits. How do I know this?

      - 1 bit for `s` (*There is one sign bit* `s`.)

- 5 bits for `exp` (*The `exp` can be any number in the range [0 .. 31].*)
- 8 bits (*It uses 8 bits.*) – 5 – 1 = 2 bits for the `frac`.

A `frac` of 2 bits is insufficient in order to exactly encode the above number using the IEEE-like encoding scheme described in this question. The number will be rounded, hence approximated.

Possible answer 2: $\mathbf{1.875 \times 2^{-15}}$

- `E = -15`
- `E = exp – bias => exp = E + bias  (bias is 15)`
- `exp = -15 + 15`
- `exp = 0 -> 00000`$_2$

An `exp` of `00000`$_2$ signifies that the number is renormalized. Therefore, it cannot be encoded (exactly or approximately) as a normalized number using the IEEE-like encoding scheme described in this question.

There were a few more acceptable answers.

<table>
<tr><th>Real Number</th><th>IEEE-like floating point number
represented using its bit pattern</th><th>Show your work</th></tr>
<tr><td>2</td><td>0 10000 00</td><td rowspan="3">Show your work for 2:
R2B(2) = 10_2
Normalized: $10.0_2 \times 2^0 = 1.00_2 \times 2^1$
V = $(-1)^0$ 1.00_2 x 2^1
s = 0_2
E = 1 -> E = exp – bias
exp = E + bias = 1 + 15 = 16
exp = 10000_2
M = 1.00_2 -> M = 1. + frac
frac = 1.00_2 – 1. = 00_2
R2B(2) = 0 10000 00</td></tr>
<tr><td>zero</td><td>0 00000 00</td></tr>
<tr><td>Largest positive denormalized</td><td>0 00000 11</td></tr>
<tr><td>$\frac{1}{4}$</td><td>0 01101 00</td><td rowspan="3">Show your work for $\frac{1}{4}$:
R2B(1/4) = 0.01_2
Normalized: $0.01_2 \times 2^0 = 1.00_2 \times 2^{-2}$
V = $(-1)^0$ 1.00_2 x 2^{-2}
s = 0_2
E = -2 -> E = exp – bias
exp = E + bias = -2 + 15 = 13
exp = 01101_2
M = 1.00_2 -> M = 1. + frac
frac = 1.00_2 – 1. = 00_2
R2B(2) = 0 01101 00</td></tr>
<tr><td>Smallest positive normalized</td><td>0 00001 00</td></tr>
<tr><td>Largest positive normalized</td><td>0 11110 11</td></tr>
</table>

7. [9 marks] Consider the following C function declaration:

```
long average( long w, long x, long y, long z);
```

which, as its name implies, calculates and returns the average of its four (4) parameters `w`, `x`, `y` and `z`. Below, implement this function in x86-64 assembly language. Make sure your x86-64 assembly code is as syntactically correct as possible, i.e., it would execute on our *target machine*. You can assume that your x86-64 assembly code will not create an overflow of any kinds.

Requirement: When answering this question, you cannot use any of the `div*` or `idiv*` x86-64 instructions.

```
# Description: Calculates and returns the average of its
#              four parameters.
# parameter to register mapping:
# w -> %rdi, x -> %rsi, y -> %rdx, z -> %rcx
   .globl    average
average:
   addq    %rdi,   %rsi      # x + w -> %rsi
   addq    %rdx,   %rsi      # x + w + y -> %rsi
   addq    %rcx,   %rsi      # x + w + y + z -> %rsi
   sarq    $2,     %rsi      # (x + w + y + z) / 4
   movq    %rsi,   %rax      # return the average
    ret
```

Marking Scheme:

- 1 mark for meaningful comments
- Total of 3 marks for computing the sum
    - 3 marks (out of 3) for correctly summing all 4 values using the proper registers (64-bit registers)

  OR

- 1 mark (out of 3) for correctly summing all 4 values using the improper registers (32-bit registers or 16-bit registers or 8-bit registers)

- Total of 3 marks for computing the average:
  - 3 marks for implementing the division without using a real number like 0.25 (these are integral registers) and without using any of the `div*` or `idiv*` x86-64 instructions.

  OR

  - 1 mark (out of 3) for implementing the division either by using a real number like 0.25 OR by using one of the `div*` or `idiv*` x86-64 instructions.

- Total of 2 marks for return value
  - 2 marks (out of 2) for returning the correct result in %rax

  OR

  - 1 mark (out of 2) for returning the correct result in %eax, %ax or %al

Table of x86-64 Registers

64-bit (quad)	32-bit (double)	16-bit (word)	8-bit (byte)		
63..0	31..0	15..0	15..8	7..0	
rax	eax	ax	ah	al	Return value
rbx	ebx	bx	bh	bl	Callee saved
rcx	ecx	cx	ch	cl	4th arg
rdx	edx	dx	dh	dl	3rd arg
rsi	esi	si		sil	2nd arg
rdi	edi	di		dil	1st arg
rbp	ebp	bp		bpl	Callee saved
rsp	esp	sp		spl	Stack pointer
r8	r8d	r8w		r8b	5th arg
r9	r9d	r9w		r9b	6th arg
r10	r10d	r10w		r10b	Caller saved
r11	r11d	r11w		r11b	Caller saved
r12	r12d	r12w		r12b	Callee saved
r13	r13d	r13w		r13b	Callee saved
r14	r14d	r14w		r14b	Callee saved
r15	r15d	r15w		r15b	Callee saved

Table of Powers of 2

Power of 2^x	Value	Power of 2^x	Value	
0	1			
1	2	-1	1/2	0.5
2	4	-2	1/4	0.25
3	8	-3	1/8	0.125
4	16	-4	1/16	0.0625
5	32	-5	1/32	0.03125
6	64	-6	1/64	0.015625
7	128	-7	1/128	0.0078125
8	256	-8	1/256	0.00390625
9	512	-9	1/512	0.001953125
10	1024	-10	1/1024	0.0009765625
11	2048			
12	4096			
13	8192			
14	16384			
15	32768			
16	65536			

Table of x86-64 Jumps

Instruction	Condition	Description
`jmp`	always	Unconditional jump
`je` / `jz`	`ZF`	Jump if equal / zero
`jne` / `jnz`	`~ZF`	Jump if not equal / not zero
`js`	`SF`	Jump if negative
`jns`	`~SF`	Jump if nonnegative
`jo`	`OF`	Jump if overflow
`jno`	`~OF`	Jump if not overflow
`jg` / `jnle`	`~(SF ^ OF) & ~ZF`	Jump if greater (signed >)
`jge` / `jnl`	`~(SF ^ OF)`	Jump if greater or equal (signed ≥)
`jl` / `jnge`	`SF ^ OF`	Jump if less (signed <)
`jle` / `jng`	`(SF ^ OF) | ZF`	Jump if less or equal (signed ≤)
`ja` / `jnbe`	`~CF & ~ZF`	Jump if greater (unsigned >)
`jae` / `jnb`	`~CF`	Jump if greater or equal (unsigned ≥)
`jb` / `jnae` / `jc`	`CF`	Jump if less (unsigned <)
`jbe` / `jna` / `jnc`	`CF | ZF`	Jump if less or equal (unsigned ≤)